Verifier-Aware Budget-Conditioned RL Decoding for Diffusion LLMs

Amin Jafarzade Team Members

April 29, 2026

Abstract

Diffusion language models generate text by gradually denoising masked sequences rather than producing tokens strictly from left to right. This generation process creates a natural sequential decision-making problem: at each denoising step, a controller can decide how aggressively to unmask tokens, whether to continue decoding, and whether suspicious tokens should be remasked. In this project, we propose to study frozen diffusion LLM decoding as a reinforcement learning problem. Our main idea is to train a lightweight RL policy that is conditioned on the remaining inference budget and enhanced with task-level verifier signals. The goal is to improve the quality–compute tradeoff during generation, especially on reasoning tasks where confidence-based decoding may keep locally confident but globally incorrect tokens.

1 Motivation

Autoregressive language models generate tokens sequentially from left to right. In contrast, masked diffusion language models begin from a masked or partially masked sequence and iteratively reconstruct the output. At each denoising step, the model predicts masked tokens, after which a decoding rule decides which predictions should be kept and which should remain masked or be remasked.

This process creates a natural RL problem. The model's confidence scores, entropy values, mask positions, and current denoising step provide state information. The controller's action can determine the unmasking ratio, confidence threshold, stopping decision, or remasking behavior. The final reward can measure both task success and inference cost.

Existing decoding strategies often rely heavily on token-level confidence. However, confidence is a local signal. In reasoning tasks, a model can be highly confident in a token that is globally wrong. For example, in a math solution, the model may confidently generate

$$48 + 24 = 62,$$

even though the computation is incorrect. A verifier-aware decoding policy could detect this inconsistency and remask the suspicious answer span.

Therefore, this project asks whether verifier signals and budget conditioning can improve diffusion LLM decoding without updating the large diffusion model itself.

2 Research Question

The main research question is:

Can a lightweight RL controller improve the quality–compute tradeoff of frozen diffusion LLM decoding by using both budget information and task-level verifier signals?

More specifically, we study the following questions:

1. Can a policy conditioned on the current iteration and remaining budget learn better decoding schedules than fixed unmasking schedules?
2. Can verifier signals help correct cases where confidence-based decoding is locally confident but globally wrong?
3. When does confidence-based decoding succeed or fail?
4. Can verifier-aware adaptive remasking improve reasoning performance under limited denoising budgets?

3 Main Idea

We keep the diffusion LLM frozen and train only a lightweight RL controller. At each denoising step, the frozen diffusion model predicts masked tokens and provides confidence scores. The controller observes these signals together with budget information and verifier features, then chooses an action controlling the next decoding step.

The controller may decide to:

- reveal a certain fraction of masked tokens;
- choose a confidence threshold;
- stop decoding early;
- remask suspicious spans, such as final-answer tokens in math problems;
- spend additional computation on uncertain examples.

The overall objective is:

$$R = \text{TaskScore} - \lambda \cdot \text{ComputeCost},$$

where the compute cost may be the number of denoising steps, number of forward passes, or wall-clock inference time.

4 Three Main Components

4.1 Idea 1: Budget-Conditioned and Iteration-Conditioned Policy

The first component is a policy that adapts its behavior based on the current denoising iteration and remaining inference budget.

Instead of using a fixed schedule such as "unmask 10% of tokens at every step," the policy observes summary statistics:

$$s_t = [\text{mean confidence}, \text{min confidence}, \text{mean entropy}, \text{mask ratio}, \text{current step}, \text{remaining budget}].$$

The action can be a discrete unmasking ratio:

$$a_t \in \{5\%, 10\%, 20\%, 40\%\}.$$

Expected behavior:

- In early steps, the policy may unmask only very confident tokens.
- In middle steps, it may reveal more tokens if uncertainty is low.
- In late steps, it may spend extra computation if many uncertain tokens remain.
- For easy examples, it may use fewer denoising steps.
- For hard examples, it may use more conservative decoding.

This turns decoding into adaptive compute allocation.

4.2 Idea 2: Verifier-Aware Remasking

The second component is the main creative part of the project. Confidence-based decoding asks:

Is the model confident in this token?

Verifier-aware decoding asks:

Is this token or span consistent with the task objective?

For math reasoning, a verifier can check:

- whether the final answer is parseable;
- whether intermediate arithmetic equations are valid;
- whether the final answer is consistent with previous computations;
- whether the answer format is correct.

For code generation, a verifier can check:

- syntax validity;
- sample test success;
- unit test pass rate.

For reversal reasoning, a verifier can check:

- exact match with the target reverse answer;
- edit distance;
- token-level F1 score.

A simple verifier-aware score can be written as:

$$\text{keep_score}_i = \alpha \cdot \text{confidence}_i + \beta \cdot \text{verifier_signal}_i.$$

If a generated span is locally confident but verifier-inconsistent, the controller can remask that span and allow the diffusion model to regenerate it.

For example:

$$12 \times 4 = 48, \quad 6 \times 4 = 24, \quad 48 + 24 = 62.$$

The token "62" may have high model confidence, but the arithmetic verifier detects an inconsistency. The controller can remask the final answer:

$$48 + 24 = \texttt{[MASK]}.$$

4.3 Idea 3: Empirical Analysis

The third component is empirical analysis. In this project, empirical analysis means systematically studying the behavior of confidence-based and verifier-aware decoding methods, rather than only reporting a final accuracy number.

We will analyze:

- when confidence-based decoding works;
- when confidence-based decoding fails;
- whether wrong answers can still have high confidence;
- whether verifier signals improve decoding;
- whether adaptive policies spend more computation on harder examples;
- whether simple heuristics perform similarly to RL policies.

This is important because the project does not need to achieve state-of-the-art results. A careful study of success and failure cases is still a valid RL research project.

5 MDP Formulation

5.1 Environment

The environment consists of:

$$\text{Frozen Diffusion LLM} + \text{Decoding Process} + \text{Task Verifier}.$$

The diffusion LLM itself is not updated. It only provides predictions, logits, confidence scores, and the next partially denoised sequence.

5.2 State

At denoising step t, the state is:

$$s_t = [r_t, m_t, c_t, h_t, b_t, v_t],$$

where:

- r_t is the current partially decoded sequence;
- m_t represents mask positions;
- c_t contains token confidence statistics;
- h_t contains entropy or uncertainty statistics;
- b_t represents current iteration and remaining budget;
- v_t contains verifier features.

A compact state representation can be:

$$s_t = [\mu_c, \min(c), \max(c), \mu_h, \rho_{\text{mask}}, t, B_{\text{remain}}, v_{\text{score}}],$$

where μ_c is mean confidence, μ_h is mean entropy, ρ_{mask} is the remaining mask ratio, and v_{score} is the verifier score.

5.3 Action

To keep the project manageable, we use a small discrete action space:

$$a_t \in \{5\%, 10\%, 20\%, 40\%, \text{stop}, \text{remask-answer-span}\}.$$

The first four actions choose how many masked tokens to unmask. The `stop` action ends decoding early. The `remask-answer-span` action remasks a suspicious span, such as a final numerical answer.

5.4 Transition

After the controller chooses action a_t, the frozen diffusion LLM performs the next denoising transition. Depending on the action, the system unmasks high-scoring tokens, keeps uncertain tokens masked, or remasks suspicious spans.

5.5 Reward

The main reward is:

$$R = \text{TaskScore} - \lambda \cdot \text{ComputeCost}.$$

For math reasoning:

$$R = \mathbf{1}[\text{correct final answer}] - \lambda \cdot \text{number of denoising steps}.$$

A shaped reward can also be used:

$$R = \mathbf{1}[\text{correct}] + \alpha \cdot \text{FormatScore} + \beta \cdot \text{ArithmeticConsistency} - \lambda \cdot \text{Steps}.$$

The shaped reward may make RL training more stable than using only sparse final correctness.

6 Proposed Methods

6.1 Baseline Decoding Methods

We first implement non-RL baselines:

- random unmasking;
- fixed-ratio unmasking;
- linear schedule unmasking;
- low-confidence remasking;
- fixed confidence-threshold decoding;
- best-of-N sampling with verifier reranking.

6.2 Budget-Conditioned RL Policy

We train a small MLP policy:

$$\pi_\phi(a_t \mid s_t),$$

where the input state contains confidence, entropy, mask ratio, timestep, and remaining budget.

The policy chooses an unmasking ratio or stopping decision. The objective is to maximize:

$$\mathbb{E}_{\pi_\phi}[\text{TaskScore} - \lambda \cdot \text{Steps}].$$

6.3 Verifier-Aware Heuristic

Before training a full RL controller, we test a simple verifier-aware heuristic:

$$\text{keep_score}_i = \alpha \cdot \log p_\theta(\hat{x}_i \mid r_t) + \beta \cdot v_i.$$

Here v_i is a token-level or span-level verifier signal. For example, if a final numerical answer is inconsistent with arithmetic computations, its verifier score is lowered, making it more likely to be remasked.

6.4 Verifier-Aware RL Policy

Finally, we train an RL policy that uses verifier features directly:

$$s_t = [\text{confidence stats}, \text{entropy stats}, \text{mask ratio}, \text{budget}, \text{verifier score}].$$

We compare:

- confidence-only RL policy;
- budget-conditioned RL policy;
- verifier-aware RL policy;
- verifier-aware heuristic decoding.

7 Experimental Plan

7.1 Experiment 1: Baseline Comparison

We compare fixed and confidence-based decoding methods. This establishes the performance of standard non-RL decoding.

Metrics:

- task accuracy;
- average number of denoising steps;
- wall-clock inference time;
- accuracy under fixed step budgets.

7.2 Experiment 2: Budget-Conditioned Decoding

We evaluate whether a budget-conditioned policy can improve the quality–compute tradeoff.

Main plot:

$$\text{Accuracy} \quad \text{vs.} \quad \text{Average Denoising Steps.}$$

We compare:

- fixed schedule;
- confidence schedule;
- budget-conditioned RL policy.

7.3 Experiment 3: Verifier-Aware Decoding

We test whether verifier-aware remasking improves reasoning outputs.

For math tasks, we compare:

- confidence-only decoding;
- verifier-aware heuristic;
- verifier-aware RL policy;
- best-of-N verifier reranking.

The important question is whether online verifier-aware remasking can outperform or match simple final reranking under the same compute budget.

7.4 Experiment 4: Confidence vs. Correctness Analysis

We measure whether confidence is a reliable predictor of correctness.

For each example, we record:

- average token confidence;
- minimum token confidence;

- entropy;
- verifier score;
- final correctness.

We then analyze whether wrong answers can still have high confidence. If yes, this supports the motivation for verifier-aware decoding.

7.5 Experiment 5: Feature Ablation

We compare policies using different state features:

Policy Features	**Purpose**
Confidence only	Tests local certainty signal
Entropy only	Tests uncertainty signal
Budget only	Tests whether policy learns a fixed schedule
Verifier only	Tests global task signal
Confidence + budget	Tests adaptive compute allocation
Confidence + verifier	Tests whether verifier improves confidence-only decoding
Full features	Tests combined policy

7.6 Experiment 6: Difficulty-Adaptive Behavior

We divide examples into easy, medium, and hard groups using initial confidence or entropy.

For each group, we report:

- average denoising steps used;
- accuracy;
- compute savings;
- improvement over confidence-only decoding.

A desirable outcome is that the policy uses fewer steps for easy examples and more careful decoding for hard examples.

7.7 Experiment 7: Failure-Case Analysis

We collect qualitative examples where:

- confidence-only decoding keeps a wrong final answer;
- verifier-aware remasking fixes the answer;
- the verifier gives a misleading signal;
- the policy stops too early;
- the policy remasks too aggressively;
- simple heuristics outperform RL.

This analysis helps explain not only whether the method works, but also why it works or fails.

8 Evaluation Metrics

8.1 Quality Metrics

- exact-match accuracy;
- multiple-choice accuracy;
- token-level accuracy;
- arithmetic consistency;
- verifier score;
- task-specific score.

8.2 Compute Metrics

- number of denoising steps;
- number of model forward passes;
- wall-clock inference time;
- average number of tokens unmasked per step;
- accuracy under fixed compute budget;
- compute required to reach target accuracy.

9 Expected Outcomes

Possible positive outcomes:

- budget-conditioned policies improve quality–compute tradeoffs;
- verifier-aware remasking improves reasoning accuracy;
- confidence-only decoding fails on high-confidence wrong answers;
- adaptive policies save compute on easy examples;
- verifier-aware policies spend more computation on hard examples.

Possible negative but useful outcomes:

- simple confidence heuristics perform similarly to RL;
- verifier-aware decoding helps only in narrow cases;
- sparse rewards make RL unstable;
- verifier errors hurt decoding;
- best-of-N reranking is stronger than online remasking.

Both positive and negative outcomes are valuable because the project emphasizes rigorous empirical analysis rather than achieving state-of-the-art performance.

10 Relation to Prior Work

This project is related to three recent directions.

First, large-scale diffusion language models such as LLaDA show that masked diffusion models can be scaled to LLM-level capabilities and can perform inference through iterative masking and remasking.

Second, RL-based post-training methods such as diffu-GRPO optimize the diffusion LLM itself for reasoning. In contrast, our project keeps the diffusion LLM frozen and trains only a lightweight controller.

Third, learned unmasking policy methods study how to improve the decoding process through RL. Our project differs by emphasizing verifier-aware reasoning signals and empirical analysis of confidence failures.

11 Conclusion

This project studies diffusion LLM decoding as a reinforcement learning problem. We propose a lightweight controller that uses confidence, entropy, remaining budget, and verifier signals to decide how aggressively to unmask or remask tokens. The central hypothesis is that confidence-only decoding may be insufficient for reasoning tasks, because local token confidence does not always imply global correctness.

The project has three main parts:

1. budget-conditioned decoding policy;
2. verifier-aware remasking;
3. empirical analysis of confidence, compute, and failure cases.

The expected contribution is not a new state-of-the-art diffusion LLM, but a careful study of whether verifier-aware and budget-conditioned control can improve the quality–compute tradeoff of frozen diffusion LLM decoding.

References

[1] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. *Large Language Diffusion Models*. 2025.

[2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. NeurIPS, 2020.

[3] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. *Structured Denoising Diffusion Models in Discrete State-Spaces*. NeurIPS, 2021.

[4] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. *MaskGIT: Masked Generative Image Transformer*. CVPR, 2022.

[5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. *Proximal Policy Optimization Algorithms*. 2017.

[6] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS, 2023.